suggest a beginner friendly project with https://imagejev.mintlify.app/api-reference/overview,python for corporate uses.

A strong beginner-friendly corporate project is a **Product Listing Photo Verification Tool**: upload a product photo plus its catalog record, and have ImageJev/imajev return structured, probability-based answers such as “Does the photo match the listed color?” and “Is the product visibly damaged?” The API is designed for image-plus-state inputs and returns typed answers with probabilities, including an explicit `unknown` option, which makes it suitable for a simple human-in-the-loop business workflow. [1] [2]

Why this project works

Corporate teams often need to validate product photos against listing data before publishing them on an e-commerce site, supplier portal, or internal catalog. This project turns that manual review into a repeatable Python workflow without requiring you to build a computer-vision model from scratch.

Component	Beginner-friendly choice
Interface	Streamlit web app
API integration	Python `requests`
Storage	CSV or SQLite
Output	Review queue with confidence scores
Business value	Fewer incorrect listings, faster QA

ImageJev/imajev accepts an image and a decision schema, then returns structured results rather than free-form generated text. Its local server accepts multipart image uploads and a JSON request containing `state` and `questions`; each question returns probabilities over the allowed answers. [2] [3] [1]

Recommended project

“Listing QA Assistant”

A reviewer uploads:

- A product photo.
- A catalog record: product name, listed color, category, condition, and optional notes.

The app asks ImageJev questions such as:

- `photo_matches_listed_color`: `yes / no / unknown`
- `product_condition`: `new / damaged / unknown`
- `photo_shows_correct_product_type`: `yes / no / unknown`
- `image_quality_acceptable`: `yes / no / unknown`

It then assigns a status:

- **Auto-approve**: all required checks pass with high confidence.
- **Manual review**: any answer is `unknown` or confidence is low.
- **Reject**: a critical check clearly fails, such as visible damage or a product-type mismatch.

This maps directly to the API's strength: it returns a probability for each option you define, including `unknown`, so your app can make conservative business decisions instead of forcing the model to guess. [1] [2]

Python starter code

```
import json
import requests

API_URL = "http://127.0.0.1:8765/v1/systemone"

listing = {
    "product_name": "Red leather office chair",
    "listed_color": "red",
    "category": "office furniture",
    "condition": "new"
}

questions = [
    {
        "id": "color_match",
        "question": "Does the photo show the listed color?",
        "options": ["yes", "no", "unknown"]
    },
    {
        "id": "product_type",
        "question": "Does the photo show the listed product type?",
        "options": ["yes", "no", "unknown"]
    },
    {
        "id": "condition",
        "question": "Does the product appear damaged?",
        "options": ["yes", "no", "unknown"]
    }
]

request_payload = {
    "state": {"listing": listing},
    "questions": questions
```

```
}

with open("chair.jpg", "rb") as photo:
    response = requests.post(
        API_URL,
        files={"image": photo},
        data={"request": json.dumps(request_payload)},
        timeout=60
    )

response.raise_for_status()
result = response.json()
print(json.dumps(result, indent=2))
```

This follows the documented request pattern: send the photo as a multipart file and the catalog data plus questions as JSON in the `request` field.[2] [1]

Simple decision logic

```
def decide(result):
    answers = result.get("answers", {})

    color = answers.get("color_match", {}).get("answer")
    product_type = answers.get("product_type", {}).get("answer")
    damaged = answers.get("condition", {}).get("answer")

    if "unknown" in (color, product_type, damaged):
        return "MANUAL_REVIEW"

    if damaged == "yes" or product_type == "no":
        return "REJECT"

    if color == "yes" and product_type == "yes" and damaged == "no":
        return "AUTO_APPROVE"

    return "MANUAL_REVIEW"
```

Adjust the exact response keys after inspecting one real API response, since field names can vary by deployment and model version.

Corporate extensions

Once the basic app works, add these practical features:

- **Review dashboard:** Show pending items, model confidence, and reviewer decisions.
- **CSV/Excel export:** Let QA teams download daily verification reports.
- **Audit trail:** Store the image hash, request payload, model response, timestamp, and final human decision.
- **Threshold configuration:** Let managers set confidence cutoffs per product category.
- **Email or Slack alert:** Notify the catalog team when rejection rates rise.

- **Batch mode:** Process a folder of supplier photos overnight.

Skills you will demonstrate

This project is excellent for a portfolio because it covers practical corporate AI skills:

- Calling a local multimodal decision API from Python.
- Designing structured business questions instead of vague prompts.
- Handling uncertainty with `unknown` and confidence thresholds.[1] [2]
- Building a usable Streamlit interface.
- Creating an auditable human-in-the-loop workflow.

A good first milestone is: **upload one photo, ask three questions, display the answers and probabilities, and label the item as approve, review, or reject.** Once that works, add CSV history and a review queue.

⁂

1. https://github.com/mohit67890/imajev
2. https://github.com/mohit67890/imajev/blob/main/README.md
3. https://pypi.org/project/imagejev/
4. https://py.imagej.net/en/latest/api.html
5. https://py.imagej.net/en/1.6.0/Install.html
6. https://py.imagej.net/en/latest/_modules/imagej.html
7. https://py.imagej.net/en/latest/Initialization.html
8. https://py.imagej.net/en/1.7.0/Initialization.html
9. https://py.imagej.net/en/latest/06-Working-with-Images.html
10. https://imagej.net/scripting/python
11. https://github.com/imagej/pyimagej/blob/main/doc/notebooks.rst
12. https://github.com/imagej/pyimagej
13. https://docs.python.org/3.15/library/
14. https://docs.python.org/2.7/
15. https://www.python.org/doc/
16. https://docs.python.org/3.13//contents.html
17. https://docs.python.org/3.14/library/
18. https://www.python.org/doc/2.5/
19. https://docs.python.org/3.13/library/index.html
20. https://docs.python.org/3/
21. https://docs.python.org/3/contents.html
22. https://docs.python.org/3.9/
23. https://github.com/mohit67890/imajev/blob/main/docs/index.html

24. https://py.imagej.net/en/stable/index.html

25. https://py.imagej.net/en/1.5.0/

26. https://py.imagej.net/en/1.7.0/notebooks.html

27. https://py.imagej.net/en/latest/index.html

28. https://py.imagej.net/en/1.5.0/Install.html

29. https://py.imagej.net/en/stable/01-Starting-PyImageJ.html